ZESPÓŁ SZKÓŁ ELEKTRYCZNO-MECHANICZNYCH

Pracownia testowania i dokumentowania aplikacji

Testowanie aplikacji

Autor: Adrian Leśnik

Klasa: 4P

Prowadzący: mgr inż. Kamil Wojnarowski

NOWY SĄCZ 2026r.

Spis treści

Wstęp 4

1. Testy jednostkowe 5

1.1 Implementacja programu 5

1.2 Implementacja testów 6

1.3 Git 7

Podsumowanie 8

2. Testy integracyjne 9

2.1 Implementacja programu 10

2.2 Implementacja testów 12

Wnioski do rozdziału 14

Wstęp

Testowanie oprogramowania polega na sprawdzaniu, czy program działa zgodnie z wymaganiami i oczekiwaniami użytkownika. Jego głównym celem jest wykrycie błędów, zanim trafią one do użytkownika końcowego.

Wyróżniamy cztery podstawowe poziomy testowania: testy jednostkowe (pojedyncze metody i klasy), integracyjne (współpraca modułów), systemowe (cały system) i akceptacyjne (zgodność z wymaganiami klienta). Testy dzielimy też na czarnoskrzynkowe, wykonywane bez znajomości kodu, oraz białoskrzynkowe, które wykorzystują wiedzę o jego budowie.

Do najważniejszych zasad testowania należą: testy pokazują obecność błędów, ale nie dowodzą ich braku, nie da się przetestować wszystkiego, a im wcześniej zaczniemy testować, tym taniej naprawimy błędy. Proces testowy obejmuje planowanie, projektowanie, wykonanie i ocenę wyników testów.

W pracy opisano testy jednostkowe na przykładzie programu napisanego w języku C# i testowanego w bibliotece xUnit.

1. Testy jednostkowe

Testy jednostkowe sprawdzają najmniejsze części programu, czyli pojedyncze metody lub klasy, w oderwaniu od reszty aplikacji. Wykonuje się je w trakcie pisania kodu, zwykle zaraz po napisaniu metody, oraz po każdej jego zmianie, aby upewnić się, że nic się nie zepsuło.

Przykład. Dla metody obliczającej cenę po rabacie test jednostkowy polega na wywołaniu jej z konkretnymi danymi i sprawdzeniu wyniku: dla ceny 100 zł i rabatu 20% oczekujemy 80 zł.

xUnit to darmowa biblioteka do testów jednostkowych na platformie .NET. Test to metoda oznaczona atrybutem `[Fact]`, a wynik sprawdza się metodami klasy `Assert`, np. `Assert.Equal` lub `Assert.Throws`. Testy uruchamia się poleceniem `dotnet test` albo w Test Explorerze w Visual Studio.

1.1 Implementacja programu

Testowany program to klasa `Sortowanie` z metodą `Sort`, która przyjmuje tablicę liczb całkowitych i zwraca nową, posortowaną rosnąco tablicę. Użyto sortowania bąbelkowego.

```
using System;

namespace Sortowanie_Przez_Wstawianie
{
    public class Sortowanie
    {
        public int[] Sort(int[] array)
        {
            if (array == null)
            {
                throw new ArgumentNullException(nameof(array));
            }

            int[] result = (int[])array.Clone();
            int n = result.Length;

            for (int i = 0; i < n - 1; i++)
            {
                for (int j = 0; j < n - i - 1; j++)
                {
                    if (result[j] > result[j + 1])
                    {
                        int temp = result[j];
                        result[j] = result[j + 1];
                        result[j + 1] = temp;
                    }
                }
            }

            return result;
        }
    }
}
```

Metoda najpierw sprawdza, czy tablica nie jest `null` – jeśli jest, zgłasza wyjątek `ArgumentNullException`. Następnie tworzy kopię tablicy, żeby nie zmieniać danych wejściowych. Dwie zagnieżdżone pętle porównują sąsiednie elementy i zamieniają je miejscami, gdy lewy jest większy od prawego. Po każdym przebiegu największy element trafia na koniec, dlatego pętla wewnętrzna jest z każdym przebiegiem krótsza. Na końcu metoda zwraca posortowaną kopię.

1.2 Implementacja testów

Testy napisano w xUnit. Klasa `SortowanieTests` zawiera cztery testy sprawdzające różne przypadki.

```
using Xunit;
using System;
using Sortowanie_Przez_Wstawianie;

namespace Sortowanie_Przez_Wstawianie_Testy
{
    public class SortowanieTests
    {
        private readonly Sortowanie _sorter = new Sortowanie();

        [Fact]
        public void Test1()
        {
            int[] input = { 4, 2, 1, 3 };
            int[] expected = { 1, 2, 3, 4 };
            Assert.Equal(expected, _sorter.Sort(input));
        }

        [Fact]
        public void Test2()
        {
            int[] input = { 1, 2, 3 };
            int[] expected = { 1, 2, 3 };
            Assert.Equal(expected, _sorter.Sort(input));
        }

        [Fact]
        public void Test3()
        {
            int[] input = Array.Empty<int>();
            int[] expected = Array.Empty<int>();
            Assert.Equal(expected, _sorter.Sort(input));
        }

        [Fact]
        public void Test4()
        {
            Assert.Throws<ArgumentNullException>(() => _sorter.Sort(null));
        }
    }
}
```

Test1 sprawdza sortowanie zwykłej tablicy, Test2 – tablicy już posortowanej, Test3 – tablicy pustej, a Test4 – czy dla wartości `null` zostanie zgłoszony wyjątek `ArgumentNullException`. Po uruchomieniu `dotnet test` wszystkie cztery testy zakończyły się wynikiem pozytywnym.

1.3 Git

Do zapisywania zmian użyto systemu Git, a repozytorium znajduje się na GitHubie. Każdy test zapisano w osobnym commicie:

- 9b2fbf8 – sprawdzenie sortowania zwykłej tablicy
- b93d938 – sprawdzenie tablicy już posortowanej
- 0ae200d – sprawdzenie pustej tablicy
- 3f986c5 – sprawdzenie rzucania wyjątku dla null

Podsumowanie

W pracy opisano testy jednostkowe i pokazano je na przykładzie metody sortującej tablicę w języku C#. Napisano cztery testy w xUnit: dla zwykłej tablicy, tablicy posortowanej, pustej oraz dla wartości null. Wszystkie zakończyły się powodzeniem, a przebieg prac zapisano w czterech commitach w Git.

2. Testy integracyjne

Testy integracyjne (ang. *integration tests*) to poziom testowania, na którym sprawdzamy, czy **kilka komponentów systemu współpracuje ze sobą poprawnie**. Test jednostkowy weryfikuje jedną odizolowaną metodę lub klasę (zależności są wtedy zazwyczaj zastępowane atrapami), natomiast test integracyjny łączy ze sobą prawdziwe obiekty i sprawdza, czy dane poprawnie przechodzą przez granice między nimi – np. od warstwy logiki biznesowej, przez kalkulator, aż do warstwy zapisu danych.

Same komponenty mogą być w pełni poprawne, a mimo to system może działać błędnie, jeśli np. jeden z modułów przekazuje drugiemu dane w innym formacie, w niewłaściwej kolejności albo jeśli błąd w jednym module pozostawia system w niespójnym stanie. Właśnie takie problemy wykrywają testy integracyjne.

Kiedy wykonujemy testy integracyjne?

- po zakończeniu testów jednostkowych poszczególnych modułów (zgodnie z układem poziomów testowania: jednostkowe → integracyjne → systemowe → akceptacyjne),
- gdy do systemu dołączany jest nowy moduł lub zmienia się sposób komunikacji między modułami,
- przed uruchomieniem testów systemowych, aby wcześnie wyłapać błędy na łączeniach między komponentami.

Porównanie z testami jednostkowymi

Cecha	Testy jednostkowe	Testy integracyjne
Zakres	jedna klasa / metoda	kilka współpracujących komponentów
Zależności	izolowane (atrapy, mocki)	prawdziwe obiekty
Szybkość	bardzo szybkie	wolniejsze
Co wykrywają	błędy w logice metody	błędy na styku komponentów

Przykład (opisowy)

W sklepie internetowym klient składa zamówienie. Moduł obsługi zamówień przyjmuje dane, przekazuje cenę do modułu odpowiedzialnego za wyliczenie rabatu, a otrzymany wynik zapisuje w bazie danych. Test jednostkowy sprawdziłby osobno samo wyliczanie rabatu. Test integracyjny wywoła natomiast złożenie zamówienia i sprawdzi, czy w bazie znalazła się pozycja z właściwą, już pomniejszoną ceną – a więc czy wszystkie trzy elementy (obsługa zamówienia, kalkulator i baza) działają razem tak, jak zakładamy.

Narzędzie: xUnit

Do napisania testów użyto biblioteki **xUnit** – popularnego frameworka testowego dla platformy .NET. xUnit służy zarówno do testów jednostkowych, jak i integracyjnych, bo to nie narzędzie, lecz *zakres* testu decyduje o jego rodzaju. Najważniejsze elementy:

- **[Fact]** – atrybut oznaczający metodę jako test,
- **Assert** – klasa z metodami sprawdzającymi wynik (`Equal`, `NotNull`, `Empty`, `Throws`),
- **dotnet test** – polecenie uruchamiające wszystkie testy w projekcie.

Testy pisane są według wzorca **AAA**: *Arrange* (przygotuj obiekty), *Act* (wykonaj akcję), *Assert* (sprawdź wynik).

2.1 Implementacja programu

Jako program testowany zaimplementowano prosty system wystawiania faktur o nazwie **SuperProstyFaktury** (język C#, .NET). System przyjmuje nazwę klienta, cenę netto i stawkę VAT, wylicza cenę brutto i zapisuje fakturę w bazie. Baza danych jest dla uproszczenia przechowywana w pamięci (lista obiektów), dzięki czemu testy nie wymagają żadnej zewnętrznej konfiguracji.

Struktura rozwiązania

```
SuperProstyFaktury.sln
 +-- SuperProstyFaktury/          (biblioteka klas - kod produkcyjny)
 |    +-- Invoice.cs
 |    +-- TaxCalculator.cs
 |    +-- InvoiceDatabase.cs
 |    +-- InvoiceService.cs
 +-- SuperProstyFaktury.Tests/    (projekt xUnit - testy)
      +-- InvoiceIntegrationTests.cs
```

Projekty utworzono poleceniami:

```
dotnet new sln -n SuperProstyFaktury
dotnet new classlib -n SuperProstyFaktury
dotnet new xunit -n SuperProstyFaktury.Tests
dotnet sln add SuperProstyFaktury SuperProstyFaktury.Tests
dotnet add SuperProstyFaktury.Tests reference SuperProstyFaktury
```

Opis działania

System składa się z czterech klas, z których każda ma jedną, jasno określoną odpowiedzialność:

- **Invoice** – model danych faktury: nazwa klienta (`Customer`) i cena brutto (`GrossPrice`).
- **TaxCalculator** – logika podatkowa. Metoda `AddVat` dolicza VAT do ceny netto według wzoru *netto* + *netto* × *VAT / 100*. Dla ujemnej stawki VAT zgłasza wyjątek

`ArgumentException`.

- **InvoiceDatabase** – uproszczona baza danych. Przechowuje zapisane faktury na liście `SavedInvoices`, a metoda `Save` dodaje do niej nową fakturę.
- **InvoiceService** – warstwa łącząca wszystko w całość. Metoda `CreateInvoice` najpierw wylicza cenę brutto w kalkulatorze, a dopiero potem zapisuje fakture w bazie. Zależności (baza i kalkulator) są przekazywane przez konstruktor.

Ważna jest kolejność operacji: ponieważ wyliczenie ceny odbywa się *przed* zapisem, błędne dane (ujemny VAT) przerywają proces jeszcze przed dotarciem do bazy i nie zostawiają w niej żadnych śmieci. Jest to zachowanie, które dobrze nadaje się do sprawdzenia w teście integracyjnym.

Kod programu

```
// Invoice.cs
namespace SuperProstyFaktury
{
    public class Invoice
    {
        public string Customer { get; set; } = "";
        public decimal GrossPrice { get; set; }
    }
}
```

```
// TaxCalculator.cs
using System;

namespace SuperProstyFaktury
{
    public class TaxCalculator
    {
        public decimal AddVat(decimal netPrice, decimal vatPercent)
        {
            if (vatPercent < 0)
                throw new ArgumentException("VAT nie moze byc ujemny");

            return netPrice + netPrice * vatPercent / 100;
        }
    }
}
```

```
// InvoiceDatabase.cs
using System.Collections.Generic;

namespace SuperProstyFaktury
{
    public class InvoiceDatabase
    {
        public List<Invoice> SavedInvoices { get; } = new();

        public void Save(Invoice invoice) => SavedInvoices.Add(invoice);
    }
}
```

```
// InvoiceService.cs
namespace SuperProstyFaktury
{
    public class InvoiceService
    {
        private readonly InvoiceDatabase _db;
        private readonly TaxCalculator _calculator;

        public InvoiceService(InvoiceDatabase db, TaxCalculator calculator)
        {
            _db = db;
            _calculator = calculator;
        }

        public void CreateInvoice(string customer, decimal netPrice,
                                  decimal vatPercent)
        {
            var gross = _calculator.AddVat(netPrice, vatPercent);
            _db.Save(new Invoice { Customer = customer, GrossPrice = gross });
        }
    }
}
```

2.2 Implementacja testów

Dla systemu napisano **cztery testy integracyjne** w xUnit. W każdym z nich tworzone są trzy *prawdziwe* obiekty – `InvoiceDatabase`, `TaxCalculator` i `InvoiceService` – połączone ze sobą tak jak w działającej aplikacji. Nie używamy atrap, dlatego test sprawdza cały przepływ: serwis → kalkulator → baza. Każdy test tworzy własną, nową bazę, więc testy są od siebie niezależne.

Opis testów

Test	Co sprawdza	Oczekiwany wynik
Integration_ShouldSaveInvoiceWithStandardVat	Faktura 1000 netto, VAT 23% przechodzi przez serwis i kalkulator i trafia do bazy.	w bazie faktura klienta „Firma ABC” z ceną brutto 1230
Integration_ShouldSaveInvoiceWithReducedVat	To samo dla obniżonej stawki VAT 8%.	w bazie faktura z ceną brutto 540 (500 + 8%)
Integration_ShouldNotSaveAnything_WhenVatIsInvalid	Ujemny VAT powoduje wyjątek w kalkulatorze, a serwis nie zapisuje nic w bazie.	zgłoszony `ArgumentException`, baza pozostaje pusta
Integration_ShouldCorrectlyAccumulateMultipleInvoicesInDatabase	Dwie faktury zapisane po kolei w tej samej bazie.	w bazie 2 faktury: 123 oraz 200

Kod testów

Plik `InvoiceIntegrationTests.cs` w projekcie `SuperProstyFaktury.Tests`:

```
using System;
using System.Linq;
using Xunit;
using SuperProstyFaktury;

namespace SuperProstyFaktury.Tests
{
    public class InvoiceIntegrationTests
    {
        [Fact]
        public void Integration_ShouldSaveInvoiceWithStandardVat()
        {
            // Arrange: laczymy wszystkie 3 komponenty w jeden system
            var db = new InvoiceDatabase();
            var calculator = new TaxCalculator();
            var service = new InvoiceService(db, calculator);

            // Act
            service.CreateInvoice("Firma ABC", 1000, 23);

            // Assert
            var saved = db.SavedInvoices.FirstOrDefault(i => i.Customer == "Firma ABC");

            Assert.NotNull(saved);
            Assert.Equal(1230m, saved.GrossPrice); // 1000 + 23%
        }
```

```
        [Fact]
        public void Integration_ShouldSaveInvoiceWithReducedVat()
        {
            // Arrange
            var db = new InvoiceDatabase();
            var calculator = new TaxCalculator();
            var service = new InvoiceService(db, calculator);

            // Act: obnizony VAT 8%
            service.CreateInvoice("Ksiegarnia", 500, 8);

            // Assert
            var saved = db.SavedInvoices.FirstOrDefault(i => i.Customer == "Ksiegarnia");

            Assert.NotNull(saved);
            Assert.Equal(540m, saved.GrossPrice); // 500 + 8%
        }
```

```
        [Fact]
        public void Integration_ShouldNotSaveAnything_WhenVatIsInvalid()
        {
            // Arrange
            var db = new InvoiceDatabase();
            var calculator = new TaxCalculator();
            var service = new InvoiceService(db, calculator);

            // Act: ujemny VAT powinien przerwac caly proces
            Assert.Throws<ArgumentException>(
                () => service.CreateInvoice("Blad Sp. z o.o.", 100, -10));

            // Assert: baza ma pozostac pusta
            Assert.Empty(db.SavedInvoices);
        }
```

```
        [Fact]
        public void Integration_ShouldCorrectlyAccumulateMultipleInvoicesInDatabase()
        {
            // Arrange
            var db = new InvoiceDatabase();
            var calculator = new TaxCalculator();
            var service = new InvoiceService(db, calculator);

            // Act: dwie faktury w tej samej bazie
            service.CreateInvoice("Klient A", 100, 23);
            service.CreateInvoice("Klient B", 200, 0);

            // Assert
            Assert.Equal(2, db.SavedInvoices.Count);

            var first = db.SavedInvoices.FirstOrDefault(i => i.Customer == "Klient A");
            var second = db.SavedInvoices.FirstOrDefault(i => i.Customer == "Klient B");

            Assert.Equal(123m, first?.GrossPrice);
            Assert.Equal(200m, second?.GrossPrice);
        }
    }
}
```

Uruchomienie testów

Testy uruchamiamy w katalogu rozwiązania poleceniem:

```
dotnet test
```

Po poprawnym wykonaniu wszystkie cztery testy powinny zostać zaliczone (*Passed: 4, Failed: 0*).

Wnioski do rozdziału

Testy integracyjne potwierdziły, że serwis, kalkulator i baza danych współpracują poprawnie: poprawne dane są przeliczane i zapisywane, a błędne dane przerywają proces bez pozostawiania śladu w bazie. Dzięki użyciu prawdziwych obiektów zamiast atrap test sprawdza realne zachowanie systemu, a nie tylko pojedynczą metodę.